

Βελτιστοποίηση - Θέματα Αριθμητικής Ανάλυσης

Ιωάννης Γ. Τσούλος

Τμήμα Πληροφορικής και τηλεπικοινωνιών
Πανεπιστήμιο Ιωαννίνων

2026

Περίληψη

- 1 Δεκαδικοί αριθμοί
 - Αναπαράσταση αριθμών
 - Αριθμητικά σφάλματα
- 2 Εύρεση ριζών
- 3 Αυτόματη διαφόριση συναρτήσεων

- ① Ένας μη μηδενικός πραγματικός αριθμός μπορεί να γραφεί στη μορφή

$$x = (-1)^s m \beta^e$$

με $s \in \{0, 1\}$, $\frac{1}{\beta} \leq m < 1$

- ② Παραδείγματα για $\beta = 10$,

$$7234.3874 = 0.72343874 \times 10^4$$

$$0.1 = 0.1 \times 10^0$$

$$-0.00452 = -0.452 \times 10^{-2}$$

- ③ Στον υπολογιστή αποθηκεύεται πεπερασμένο πλήθος ψηφίων του m και του e . Αυτό σημαίνει πως δεν υπάρχει πάντα ακριβής αποθήκευση ενός αριθμού.

Μορφή κ-ψηφίων

- Έστω η βάση του συστήματος είναι β
- Κάθε αριθμός δίνεται από την σχέση:
$$fl_k(x) = (-1)^s \times (0.d_1d_2 \dots d_k)_\beta \times \beta^e \text{ με}$$
$$s \in \{0, 1\}, 0 \leq d_i < \beta$$
- k είναι το πλήθος των ψηφίων, s είναι το πρόσημο του αριθμού, e εκθέτης του αριθμού
- **Παράδειγμα 1:**
$$123.245 = fl_6(123.245) = (-1)^0 \times (0.123245) \times 10^3$$
- **Παράδειγμα 2:** $fl_6(\sqrt{2}) = (-1)^0 \times (0.141421) \times 10^1$ (δεν εμφανίζονται όλα τα ψηφία)
- **Παράδειγμα 3:** $fl_6(-9876543) = (-1)^1 \times (0.987654) \times 10^7$

Αποκοπή και στρογγυλοποίηση

- ❶ Όταν ένας αριθμός δεν μπορεί να αναπαρασταθεί με k σημαντικά ψηφία, πρέπει να προσεγγιστεί.
- ❷ **Αποκοπή** (chopping): Τα ψηφία μετά το k -οστό σημαντικό ψηφίο απορρίπτονται. Πχ αν $x = 1.23456789$, $fl_5(x) = 1.2345$
- ❸ **Στρογγυλοποίηση** (rounding): Εξετάζεται το πρώτο ψηφίο που απορρίπτεται. Για 5 ψηφία πχ, αν είναι ≥ 5 , το τελευταίο ψηφίο που διατηρείται αυξάνεται κατά μία μονάδα. Παράδειγμα: $x = 1.23456789$, $fl_5(x) = 1.2346$
- ❹ Συνήθως η στρογγυλοποίηση οδηγεί σε μικρότερα σφάλματα.
- ❺ Παράδειγμα: $x = 12.3451$, Για $k = 4$, αποκοπή: 12.34, στρογγυλοποίηση: 12.35

IEEE αριθμοί

- 1 Οι αριθμοί αναπαρίστανται σαν
$$x = sign \times fraction \times base^{exponent}$$
- 2 sign είναι το πρόσημο του αριθμού
- 3 fraction είναι τα ψηφία του αριθμού
- 4 base είναι η βάση του αριθμητικού συστήματος πχ 2 ή 10.
- 5 exponent εκθέτης στο αριθμητικό σύστημα

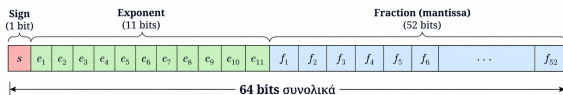
Μορφές κινητής υποδιαστολής IEEE 754

- Υπάρχουν τρεις τύποι δεκαδικών τιμών
- Στην συνέχεια εμφανίζονται οι διαστάσεις τους σε bits

Τύπος	Συνολικό μήκος	Πρόσημο	Εκθέτης	κλασματικό μέρος
Single	32	1	8	23
Double	64	1	11	52
Quadruple	128	1	15	112

- Ο τύπος double:

IEEE 754 Double Precision (64 bits)



$$x = (-1)^s (1.f)_2 2^E$$

Ακρίβεια αριθμών κινητής υποδιαστολής

Τύπος	Ακρίβεια (bits)	e_m
Single	24	$2^{-23} \simeq 10^{-7}$
Double	53	$2^{-52} \simeq 2.2 \times 10^{-16}$
Quadruple	113	2^{-112}

- ❶ Το NaN (Not a Number) είναι ειδική τιμή κινητής υποδιαστολής που δηλώνει ότι το αποτέλεσμα μιας αριθμητικής πράξης δεν είναι έγκυρος πραγματικός αριθμός.
- ❷ Η συνάρτηση `isnan()` χρησιμοποιείται στις περισσότερες γλώσσες για να μάθουμε αν ένας αριθμός είναι αόριστος. Πχ, το κλάσμα $0/0$ δεν ορίζεται.
- ❸ Η συνάρτηση `isinf()` χρησιμοποιείται όταν ένας αριθμός αν και υπάρχει δεν μπορεί να αναπαρασταθεί σε υπολογιστή.
- ❹ Αν και μπορούν να χρησιμοποιηθούν και σε ακέραιες εκφράσεις, συνήθως χρησιμοποιούνται σε δεκαδικές τιμές.

Προγραμματιστικό παράδειγμα

```
#include <cmath>

if (std::isnan(x))
    std::cout << "NaN";

if (std::isinf(x))
    std::cout << "Infinity";

if (std::isfinite(x))
    std::cout << "Finite";
```

Ορισμοί

- **Σφάλμα αριθμού:** $error(x_{eval}) = x_{true} - x_{eval}$
- **Απόλυτο σφάλμα:** $abserror(x_{eval}) = \|error(x_{eval})\| = \|x_{true} - x_{eval}\|$
- **Σχετικό Σφάλμα:** $relerror(x_{eval}) = \frac{x_{true} - x_{eval}}{x_{true}}$
- Προσέγγιση αριθμού με k **σημαντικά** ψηφία αν ισχύει:
 $abserror(x_{eval}) \leq \frac{1}{2} \times 10^k$

Δείκτης κατάστασης συνάρτησης

- Δείχνει αν οι υπολογισμοί είναι ευσταθείς για μια συνάρτηση
- Δίνεται από: $E = \left| \frac{f'(x_{true})}{f(x_{true})} x_{true} \right|$
- Παράδειγμα 1: $f(x) = x\sqrt{x}$. Δείκτης κατάσταση $3/2$
- Παράδειγμα 2: $f(x) = \frac{1}{x^2-4}$. Έχει δείκτη κατάστασης $\frac{2x^2}{x^2-4}$, που σημαίνει ότι κοντά στο ± 2 ο δείκτης κατάστασης παίρνει σχεδόν άπειρες τιμές.

Υπολογισμός ϵ

Έστω ο αλγόριθμος

```
#include <iostream>
```

```
#include <iomanip>
```

```
int main()
```

```
{
```

```
    double eps = 1.0;
```

```
    while (1.0 + eps / 2.0 > 1.0)
```

```
    {
```

```
        eps = eps / 2.0;
```

```
    }
```

```
    std::cout << std::setprecision(20);
```

```
    std::cout << "Machine_epsilon=" << eps << std::endl;
```

```
    return 0;
```

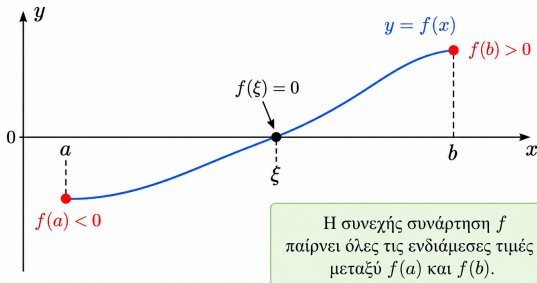
```
}
```

Θεώρημα Bolzano

- Αν $y = f(x)$ συνεχής στο $[a, b]$ και ισχύει $f(a)f(b) < 0$, τότε υπάρχει $\xi \in [a, b]$ τέτοιο ώστε $f(\xi) = 0$
- Αν η $f(x)$ διατηρεί την μονοτονία της στο $[a, b]$, τότε αυτή η ρίζα είναι και μοναδική.

Θεώρημα Bolzano

Αν f είναι συνεχής στο $[a, b]$ και $f(a)f(b) < 0$, τότε υπάρχει $\xi \in (a, b)$ τέτοιο ώστε $f(\xi) = 0$.

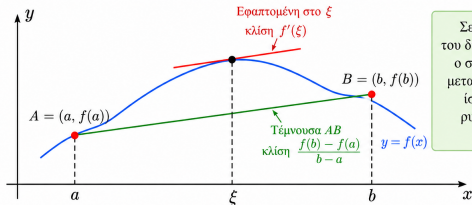


Θεώρημα μέσης τιμής

Θεώρημα Μέσης Τιμής

Αν η f είναι συνεχής στο $[a, b]$ και παραγωγίσιμη στο (a, b) , τότε υπάρχει τουλάχιστον ένα $\xi \in (a, b)$ τέτοιο ώστε

$$f'(\xi) = \frac{f(b) - f(a)}{b - a}$$



Σε κάποιο σημείο ξ του διαστήματος (a, b) , ο στιγμιαίος ρυθμός μεταβολής της f είναι ίσος με τον μέσο ρυθμό μεταβολής στο $[a, b]$.

Υπάρχει $\xi \in (a, b)$ τέτοιο ώστε η εφαπτομένη στην καμπύλη $y = f(x)$ να είναι **παράλληλη** με την ευθεία που ενώνει τα σημεία A και B .

Μέθοδος διχοτόμησης

- Εφαρμογή του θεωρήματος bolzano



Μέθοδος Διχοτόμησης

Δεδομένης μιας συνεχούς συνάρτησης f στο $[a, b]$ με $f(a)f(b) < 0$, η μέθοδος διχοτόμησης βρίσκει μια ρίζα της f στο διάστημα $[a, b]$ με διαδοχική υποδιάρθρωση του διαστήματος.

Αλγόριθμος (C++)

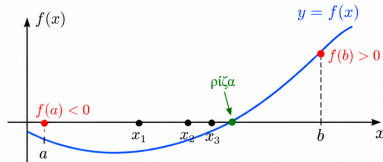
```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <functional>

double bisection(const std::function<double(double)>& f,
    double a, double b, double tol, int maxIter,
    int& iterations, bool& converged)
{
    double fa = f(a);
    double fb = f(b);
    if (fa * fb > 0) {
        std::cerr << "Error: f(a) and f(b) must have opposite signs.\n";
        converged = false;
        iterations = 0;
        return std::numeric_limits<double>::quiet_NaN();
    }

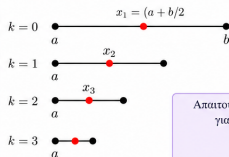
    iterations = 0;
    converged = false;
    double x = a;

    for (iterations = 1; iterations <= maxIter; ++iterations) {
        x = 0.5 * (a + b);
        double fx = f(x);
        // Κριτήριο τερματισμού
        if (std::abs(fx) < tol || 0.5 * (b - a) < tol) {
            converged = true;
            return x;
        }
        // Επιλογή υποδιαστήματος
        if (fa * fx < 0) {
            b = x;
            fb = fx;
        } else {
            a = x;
            fa = fx;
        }
    }
    return x; // καλύτερη τρέχουσα προσέγγιση
}
```

Γραφική απεικόνιση



Διαδοχικές υποδιαιρέσεις του διαστήματος



Σε κάθε επανάληψη το μήκος του διαστήματος υποδιπλασιάζεται:

$$L_k = \frac{b-a}{2^k}$$

Απαιτούμενος αριθμός επαναλήψεων για μήκος διαστήματος $\leq \epsilon$:

$$k \geq \left\lceil \log_2 \frac{b-a}{\epsilon} \right\rceil$$

Παράδειγμα

$f(x) = x^3 - x - 2$, $[a, b] = [1, 2]$
 $f(1) = -2$, $f(2) = 4 \Rightarrow f(1)f(2) < 0$

Πρώτες επαναλήψεις:

$$x_1 = 1.5, \quad f(1.5) = -0.125 \Rightarrow [1, 2] \rightarrow [1.5, 2]$$

$$x_2 = 1.75, \quad f(1.75) = 1.609... \Rightarrow [1.5, 2] \rightarrow [1.5, 1.75]$$

$$x_3 = 1.625, \quad f(1.625) = 0.666... \Rightarrow [1.5, 1.75] \rightarrow [1.5, 1.625]$$

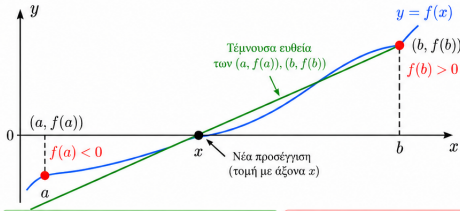
Η μέθοδος διχοτόμησης είναι εγγεγραμμένα συγκλίνουσα, αλλά συνήθως **αργή** (γραμμική σύγκλιση).

Μέθοδος εσφαλμένης θέσης

Μέθοδος Εσφαλμένης Θέσης (False Position)

Δεδομένης μιας συνεχούς συνάρτησης f στο $[a, b]$ με $f(a)f(b) < 0$, η μέθοδος εσφαλμένης θέσης βρίσκει μια ρίζα της f στο διάστημα $[a, b]$ χρησιμοποιώντας την ευθεία που ενώνει τα σημεία $(a, f(a))$ και $(b, f(b))$.

$$x = b - f(b) \frac{b - a}{f(b) - f(a)} = \frac{af(b) - bf(a)}{f(b) - f(a)}$$



Αλγόριθμος

- Επιλέγουμε a, b έτσι ώστε $f(a)f(b) < 0$.
- Υπολογίζουμε $x = \frac{af(b) - bf(a)}{f(b) - f(a)}$.
- Αν $|f(x)| < \text{tol}$ ή $|b - a| < \text{tol} \rightarrow$ τέλος.
- Αν $f(a)f(x) < 0 \rightarrow b = x$, αλλιώς $a = x$.
- Επανάληψη.

Πλεονεκτήματα

- Χρησιμοποιεί και τις τιμές $f(a), f(b)$, όχι μόνο τα πρόσμμά τους.
- Διατηρεί πάντα διάστημα που περιέχει ρίζα (εγγυημένη σύγκλιση για συνεχή f).

Διαδοχικές προσεγγίσεις

$$k=0 \quad \begin{array}{c} \bullet \\ a \end{array} \quad \begin{array}{c} \bullet \\ x_1 \end{array} \quad \begin{array}{c} \bullet \\ b \end{array} \quad x_1 = \frac{af(b) - bf(a)}{f(b) - f(a)}$$

$$k=1 \quad \begin{array}{c} \bullet \\ a \end{array} \quad \begin{array}{c} \bullet \\ x_2 \end{array} \quad \begin{array}{c} \bullet \\ b_1 \end{array} \quad x_2 = \frac{af(b_1) - b_1f(a)}{f(b_1) - f(a)}$$

$$k=2 \quad \begin{array}{c} \bullet \\ a \end{array} \quad \begin{array}{c} \bullet \\ x_3 \end{array} \quad \begin{array}{c} \bullet \\ b_2 \end{array} \quad x_3 = \frac{af(b_2) - b_2f(a)}{f(b_2) - f(a)}$$

$$k=3 \quad \begin{array}{c} \bullet \\ a \end{array} \quad \begin{array}{c} \bullet \\ x_4 \end{array} \quad \begin{array}{c} \bullet \\ b_3 \end{array}$$

Σε κάθε επανάληψη διατηρείται υποδιάστημα $[a, b]$ στο οποίο υπάρχει ρίζα (αλλαγή προσήμου).

Μειονεκτήματα

- Μπορεί να παρουσιάσει στασιμότητα (ένα άκρο τα μένει σχεδόν σταθερό για πολλές επαναλήψεις).
- Συνήθως συγκλίνει πιο αργά από τη μέθοδο του Newton (όταν αυτή είναι εφαρμόσιμη).
- Δεν είναι πάντα σημαντικά ταχύτερη από τη διχοτόμηση.

Μέθοδος Newton

Μέθοδος Newton (Newton-Raphson)

Δεδομένης μιας παραγωγίσιμης συνάρτησης f και μιας αρχικής προσέγγισης x_0 , η μέθοδος Newton παράγει μια ακολουθία $\{x_k\}$ με τύπο

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

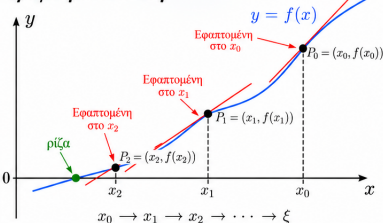
Εφόσον η x_0 είναι κατάλληλη και $f'(x) \neq 0$ κοντά στη ρίζα, η μέθοδος συγκλίνει συνήθως πολύ γρήγορα (τετραγωνική σύγκλιση για απλή ρίζα).

Αλγόριθμος (C++)

```
#include <iostream>
#include <cmath>
#include <functional>

double newton(const std::function<double(double)>& f,
              const std::function<double(double)>& df,
              double x0, double tol, int maxIter,
              int& iterations, bool& converged)
{
    double x = x0;
    iterations = 0;
    converged = false;
    for (iterations = 1; iterations <= maxIter; ++iterations) {
        double fx = f(x);
        double dfx = df(x);
        if (std::abs(fx) < tol) {
            converged = true;
            return x;
        }
        if (std::abs(dfx) < 1e-14) {
            std::cerr << "Error: derivative near zero.\n";
            return std::numeric_limits<double>::quiet_NaN();
        }
        x = x - fx / dfx;
    }
    return x; // καλύτερη διαθέσιμη προσέγγιση
}
```

Γραφική απεικόνιση



Παράδειγμα

$$f(x) = x^3 - x - 2, \quad f'(x) = 3x^2 - 1$$

Αρχική τιμή: $x_0 = 1.5$

k	x_k	$f(x_k)$
0	1.500000	-0.125000
1	1.521739	0.002137
2	1.521380	0.000001
3	1.521380	~ 0

Ρίζα: $\xi \approx 1.5213797068$

Ρυθμός σύγκλισης (απλή ρίζα)

$$|e_{k+1}| \approx C|e_k|^2 \quad (\text{τετραγωνική σύγκλιση})$$

Πλεονεκτήματα

- Πολύ γρήγορη τοπική σύγκλιση (τετραγωνική για απλή ρίζα).
- Χρησιμοποιεί και την τιμή της παραγώγου.
- Απλή υλοποίηση.

Μειονεκτήματα

- Δεν είναι εγγυημένη η σύγκλιση.
- Μπορεί να αποκλίνει για κακή επιλογή x_0 .
- Μπορεί να αποτύχει αν $f'(x_k) \approx 0$.
- Μπορεί να συγκλίνει σε διαφορετική ρίζα ανάλογα με το x_0 .
- Απαιτεί τον υπολογισμό της $f'(x)$.

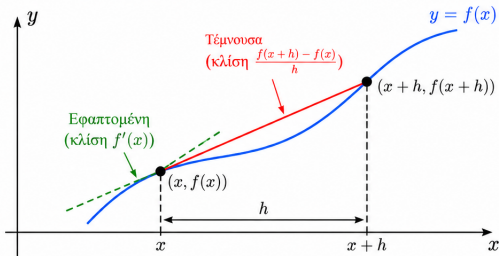
Κλειδί επιτυχίας:
καλή αρχική προσέγγιση!

Παραγωγή με διαφορές

- Η παράγωγος πολλές φορές δύσκολα υπολογίζεται.
- Μπορεί να υπολογιστεί έμμεσα με πεπερασμένες διαφορές
- Απαιτήση: η συνάρτηση να είναι συνεχής

Αριθμητική παραγωγή — Πεπερασμένες διαφορές

Όταν η αναλυτική παράγωγος μιας συνάρτησης δεν είναι διαθέσιμη ή είναι δύσκολο να υπολογιστεί, μπορούμε να την προσεγγίσουμε χρησιμοποιώντας πεπερασμένες διαφορές (finite differences).



Από τον ορισμό της παραγώγου

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Πρόσθια διαφορά (forward difference)

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

για μικρό $h > 0$.

Προϋπόθεση

Η συνάρτηση f πρέπει να είναι παραγωγίσιμη και επαρκώς ομαλή στην περιοχή του x .

Καθώς το h μικραίνει...

η τέμνουσα προσεγγίζει την εφαπτομένη και η κλίση $\frac{f(x+h) - f(x)}{h}$ προσεγγίζει την πραγματική παράγωγο $f'(x)$.

Η επιλογή του h είναι κρίσιμη

- Μεγάλο $h \rightarrow$ μεγαλύτερο σφάλμα προσέγγισης (truncation error).
- Πολύ μικρό $h \rightarrow$ σφάλμα στρογγυλοποίησης και καταστροφικής αφαίρεσης (round-off error).

Συμπέρασμα

Επιλέγουμε ένα κατάλληλο μικρό h (π.χ. με βάση το machine epsilon), ώστε να ισορροπούμε ανάμεσα στα δύο είδη σφάλματος.

Παράγωγος με διαφορές

Τύποι Πεπερασμένων Διαφορών — Πρώτες Παράγωγοι

Οι πεπερασμένες διαφορές (finite differences) χρησιμοποιούν τιμές της συνάρτησης σε γειτονικά σημεία για να προσεγγίσουν τις παραγώγους της. Οι τύποι που ακολουθούν προκύπτουν από ανάπτυγμα Taylor και ισχύουν όταν η συνάρτηση είναι επαρκώς ομαλή στην περιοχή του σημείου.

Σημαντική παρατήρηση

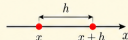
Η επιλογή του βήματος h (ή k) είναι κρίσιμη: πολύ μεγάλο βήμα $\rightarrow$ μεγάλο σφάλμα προσέγγισης, πολύ μικρό βήμα $\rightarrow$ σφάλμα στρωγγυλοποίησης. Συνήθως επιλέγουμε $h \approx \sqrt{\epsilon} \cdot \max(1, |x|)$, όπου ϵ το machine epsilon.

Πρώτη παράγωγος (μονοδιάστατη)

Πρόσθα διαφορά (forward difference)

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

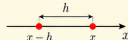
Σφάλμα: $O(h)$



Οπίσθια διαφορά (backward difference)

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}$$

Σφάλμα: $O(h)$



Κεντρική διαφορά (central difference)

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Σφάλμα: $O(h^2)$

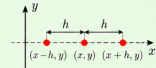


Πρώτες μερικές παράγωγοι (διδιάστατη συνάρτηση $f(x, y)$)

Παράγωγος ως προς x

$$f_x(x, y) \approx \frac{f(x+h, y) - f(x-h, y)}{2h}$$

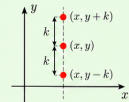
Σφάλμα: $O(h^2)$



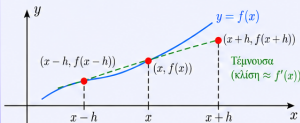
Παράγωγος ως προς y

$$f_y(x, y) \approx \frac{f(x, y+k) - f(x, y-k)}{2k}$$

Σφάλμα: $O(k^2)$



Γεωμετρική ερμηνεία (κεντρική διαφορά)



Παράδειγμα

$$f(x) = \sin(x), \quad x = 1, \quad h = 0.1$$

Τύπος	Προσέγγιση	Σφάλμα
Πρόσθα	0.497502	$5.8 \cdot 10^{-3}$
Οπίσθια	0.488356	$3.3 \cdot 10^{-3}$
Κεντρική	0.495838	$8.3 \cdot 10^{-6}$
Ακριβής	$\cos(1) = 0.540302$	—

Συμπεράσματα

- Η κεντρική διαφορά είναι σημαντικά ακριβέστερη ($O(h^2)$) από την πρόσθα και οπίσθια ($O(h)$).
- Οι τύποι επεκτείνονται εύκολα σε πολυδιάστατες συναρτήσεις.
- Η σωστή επιλογή του βήματος h (ή k) είναι κρίσιμη για την ακρίβεια.
- Οι πεπερασμένες διαφορές είναι απλές στην υλοποίηση και δεν απαιτούν αναλυτικές παραγώγους.

Η μέθοδος Secant

Μέθοδος Τέμνουσας (Secant Method)

Η μέθοδος της τέμνουσας είναι μια ανοικτή μέθοδος για τον υπολογισμό ρίζας μιας συνάρτησης f , που χρησιμοποιεί δύο προηγούμενες προσεγγίσεις και την ευθεία που περνά από τα σημεία $(x_{k-1}, f(x_{k-1}))$ και $(x_k, f(x_k))$. Δεν απαιτεί τον υπολογισμό της παραγώγου $f'(x)$.

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}$$

Αλγόριθμος

1. Επιλέγουμε δύο αρχικές προσεγγίσεις x_0, x_1 .
2. Για $k = 1, 2, \dots, \text{maxIter}$

(α) Υπολογίζουμε

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

(β) Αν $|f(x_{k+1})| < \text{tol}$ ή $|x_{k+1} - x_k| < \text{tol}$
τότε τερματίζουμε.

(γ) Θέτουμε $x_{k-1} = x_k$, $x_k = x_{k+1}$.

3. Επιστρέφουμε την προσέγγιση x_{k+1} .

Γεωμετρική ερμηνεία

Η επόμενη προσέγγιση x_{k+1} είναι το σημείο τομής της ευθείας που περνά από $(x_{k-1}, f(x_{k-1}))$ και $(x_k, f(x_k))$ με τον άξονα x .

Παράδειγμα

$$f(x) = x^3 - x - 2$$

Αρχικές τιμές: $x_0 = 1, \quad x_1 = 2$

k	x_k	$f(x_k)$
0	1.000000	-2.000000
1	2.000000	4.000000
2	1.333333	-0.962963
3	1.462687	-0.333702
4	1.531169	0.034764
5	1.521380	0.000285
6	1.521379	-0.000001

$$\text{Pířα: } \varepsilon \approx 1.5213797068$$

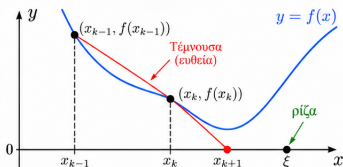
Ρυθμός σύγκλισης

Η μέθοδος της τέμνουσας έχει υπεργραμμική σύγκλιση με τάξη

$$p = \frac{1 + \sqrt{5}}{2} \approx 1.618$$

(χρυσός λόγος).

Γραφική απεικόνιση



Πλεονεκτήματα

- Δεν απαιτεί τον υπολογισμό της παραγώγου $f'(x)$.
- Απλούστερη από τη μέθοδο Newton.
- Συνήθως γρηγορότερη από τη διχοτόμηση (όταν συγκλίνει).

Μειονεκτήματα

- Δεν είναι εγγυημένη η σύγκλιση.
- Μπορεί να αποκλίνει για κακή επιλογή αρχικών τιμών.
- Μπορεί να συγκλίνει σε διαφορετική ρίζα.
- Αποτυγχάνει αν $f(x_k) \approx f(x_{k-1})$ (διαίρεση με μηδέν).
- Γενικά πιο αργή από τη Newton (τοπικά τετραγωνική σύγκλιση, τάξη ≈ 1.618).

Σύγκριση με άλλες μεθόδους

- **Newton:** ταχύτερη (τετραγωνική σύγκλιση), αλλά απαιτεί $f'(x)$.
- **Τέμνουσα:** δεν απαιτεί $f'(x)$, τάξη ≈ 1.618 .
- **Διχοτόμηση:** εγγυημένη σύγκλιση, αλλά γραμμική (πιο αργή).

Δυσκολίες στην παραγωγή

- Πολλές φορές η παράγωγος υπολογίζεται δύσκολα
- Σε κάποιες περιπτώσεις είναι τεχνικά αδύνατον να υπολογιστεί (πχ αν μεσολαβεί κάποιο άλλο βήμα με άνοιγμα κάποιου αρχείου)
- Υπάρχουν βιβλιοθήκες προγραμματισμού που κάνουν πιο εύκολη την παραπάνω εργασία.
- Συνήθως απαιτούν διατύπωση της συνάρτησης με διαφορετικό τρόπο ή χρησιμοποιούν διαφορετικούς τύπους δεδομένων.

- Είναι γραμμένη σε C++
- Διαθέσιμη από τον ιστοχώρο
`https://github.com/autodiff/autodiff`
- Ο προγραμματιστής πρέπει να χρησιμοποιήσει διαφορετικό τρόπο για να γράψει τις συναρτήσεις
- Απαιτεί την χρήση διαφορετικών τύπων δεδομένων.

Παράδειγμα γραφής συνάρτησης στο AutoDiff

```
#include <iostream>
#include <autodiff/forward/dual.hpp>
using namespace autodiff;
dual f(dual x)
{
    return x * x * sin(x);
}
int main()
{
    dual x = 2.0;
    double dfdx =derivative(f, wrt(x), at(x));
    std::cout << "f'(2) = " << dfdx << '\n';
    return 0;
}
```

Παρατηρήσεις

- Οι συναρτήσεις μπορούν να έχουν οποιοδήποτε αριθμό ορισμάτων
- Ο τύπος `double` πρέπει να αντικατασταθεί από το **dual**.

- Η αρχική συνάρτηση παρέχεται σε συμβολική μορφή.
- Μεσολαβεί ένας διερμηνευτής που βρίσκει τα σύμβολα της συνάρτησης
- Το τελικό αποτέλεσμα είναι σε συμβολική μορφή.
- Παράδειγμα: η βιβλιοθήκη Ev3 διαθέσιμη από <https://github.com/openturns/ev3>

Παράδειγμα χρήσης της Ev3

Παράδειγμα Συμβολικής Διαφορίσης με Ev3

Συνάρτηση

$$f(x_1, x_2) = x_1 \sin(x_2)$$

Αναλυτικές παράγωγοι

$$\frac{\partial f}{\partial x_1} = \sin(x_2)$$

$$\frac{\partial f}{\partial x_2} = x_1 \cos(x_2)$$

Gradient

$$\nabla f(x_1, x_2) = \begin{pmatrix} \sin(x_2) \\ x_1 \cos(x_2) \end{pmatrix}$$

Έξοδος προγράμματος

$$\begin{aligned} f(x_1, x_2) &= x_1 \sin(x_2) \\ df/dx_1 &= \sin(x_2) \\ df/dx_2 &= x_1 \cos(x_2) \end{aligned}$$

Κώδικας C++ με Ev3

```
#include <iostream>
#include <ev3/Expression.h>
#include <ev3/ExpressionParser.h>

int main()
{
    Ev3::ExpressionParser parser;
    parser.SetVariableID("x1", 0);           // Αντιστοίχιση μεταβλητών σε IDs
    parser.SetVariableID("x2", 1);           // x1 έχει ID 0
                                           // x2 έχει ID 1

    int nerr = 0;
    Ev3::Expression expr = parser.Parse("x1*sin(x2)", nerr); // f(x1, x2) = x1*sin(x2)

    Ev3::Expression dx1 = Ev3::Diff(expr, 0); // df/dx1 (ID 0)
    Ev3::Expression dx2 = Ev3::Diff(expr, 1); // df/dx2 (ID 1)

    std::cout << "f(x1,x2) = " << expr->ToString() << std::endl;
    std::cout << "df/dx1" << " = " << dx1->ToString() << std::endl;
    std::cout << "df/dx2" << " = " << dx2->ToString() << std::endl;

    return 0;
}
```

Τι σημαίνει το Diff(expr, i);

- Το i είναι το ID της μεταβλητής ως προς την οποία παίρνουμε παράγωγο.
- Στο παράδειγμα: $x_1 \rightarrow \text{ID } 0$, $x_2 \rightarrow \text{ID } 1$.
- $\text{Ev3::Diff}(\text{expr}, 0) \rightarrow \frac{\partial f}{\partial x_1}$
- $\text{Ev3::Diff}(\text{expr}, 1) \rightarrow \frac{\partial f}{\partial x_2}$

Σημεία προσοχής

- Πρέπει να ορίσουμε τα IDs των μεταβλητών με `parser.SetVariableID()`.
- Η συνάρτηση δίνεται ως συμβολισμός και αναλύεται από τον parser.
- Η Ev3 επιστρέφει τον συμβολικό τύπο της παραγώγου, όχι αριθμητική τιμή.
- Το ίδιο σήμα επεκτείνεται εύκολα σε περισσότερες μεταβλητές.

Σύνοψη

- Παρουσιάστηκαν βασικές έννοιες μαθηματικών
- Παρουσιάστηκαν έννοιες μετατροπής αριθμών.